

SIMATS ENGINEERING ASSESSMENT TOOL 2

NAME: K.DEEPIKA

REG NO: 192371042

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4303

C03-AT2

1. Servlet Architecture &

Workflow Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

Answer:

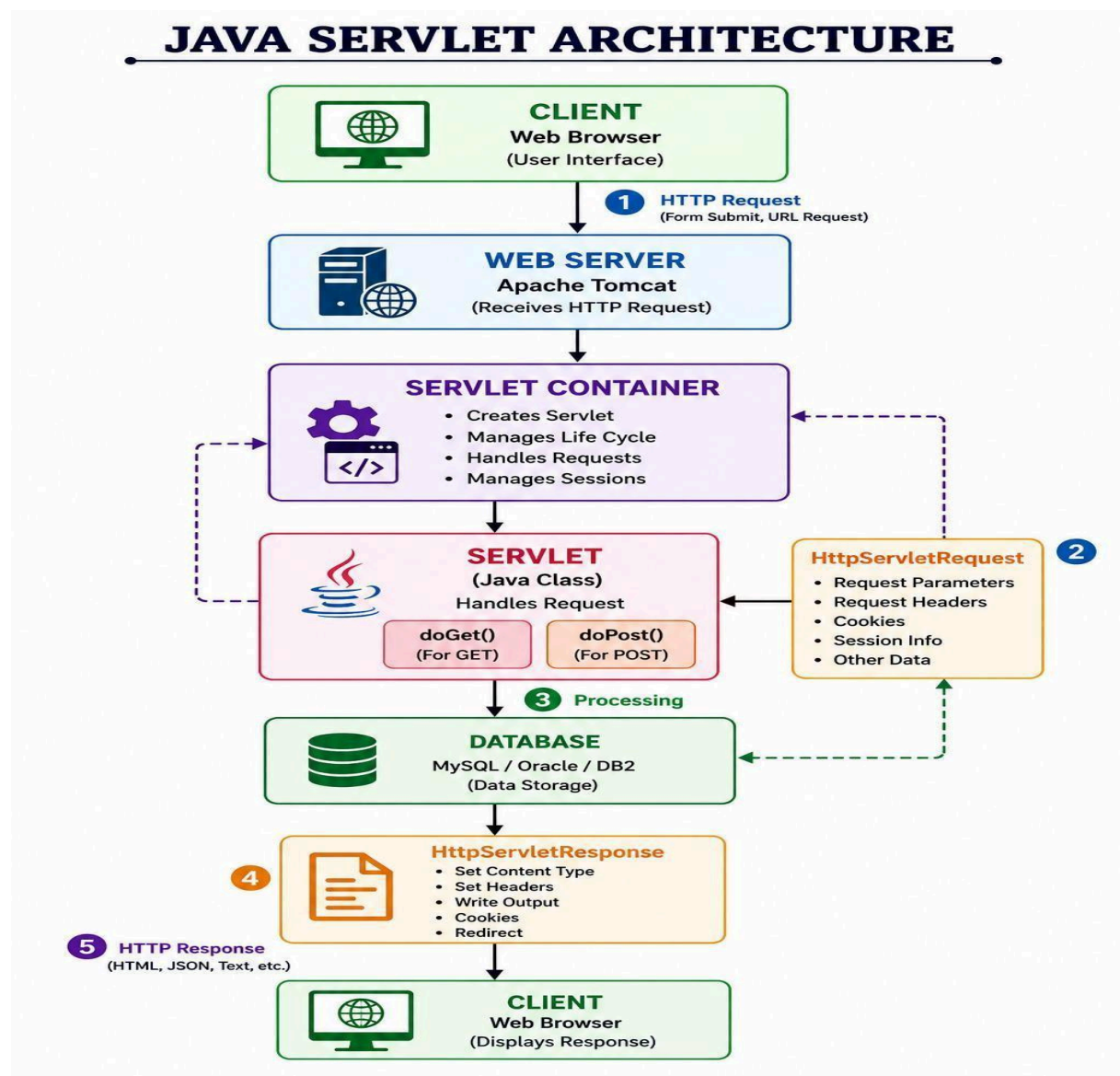
Java Servlets – Complete Solutions

1. Servlet Architecture &

Workflow Definition

A **Java Servlet** is a server-side Java program used to handle client requests and generate dynamic responses.

Servlet Architecture



Request–Response Cycle

1. User enters information in a web browser.
2. Browser sends an **HTTP request** to the web server.
3. The web server passes the request to the **Servlet Container**.
4. The container identifies the required servlet.
5. It creates `HttpServletRequest` and `HttpServletResponse` objects.
6. The servlet processes the request.
7. The servlet may communicate with a database or other services.
8. The servlet generates a response.
9. The container sends the HTTP response back to the browser.
10. Browser displays the result.

Request Object

HttpServletRequest contains information sent by the client.

Example:

```
String name = request.getParameter("name");
```

It can obtain:

- Form parameters
- Request method
- Headers
- Cookies
- Session information

Response Object

HttpServletResponse is used to send data back to the client.

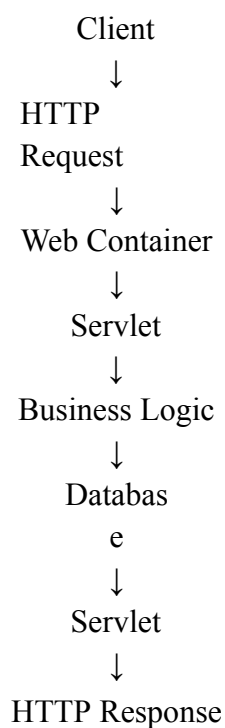
Example:

```
response.setContentType("text/html");
```

```
PrintWriter out = response.getWriter();
```

```
out.println("<h1>Welcome!</h1>");
```

Simple Workflow



↓
Client

2. Servlet Life Cycle

Analysis Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

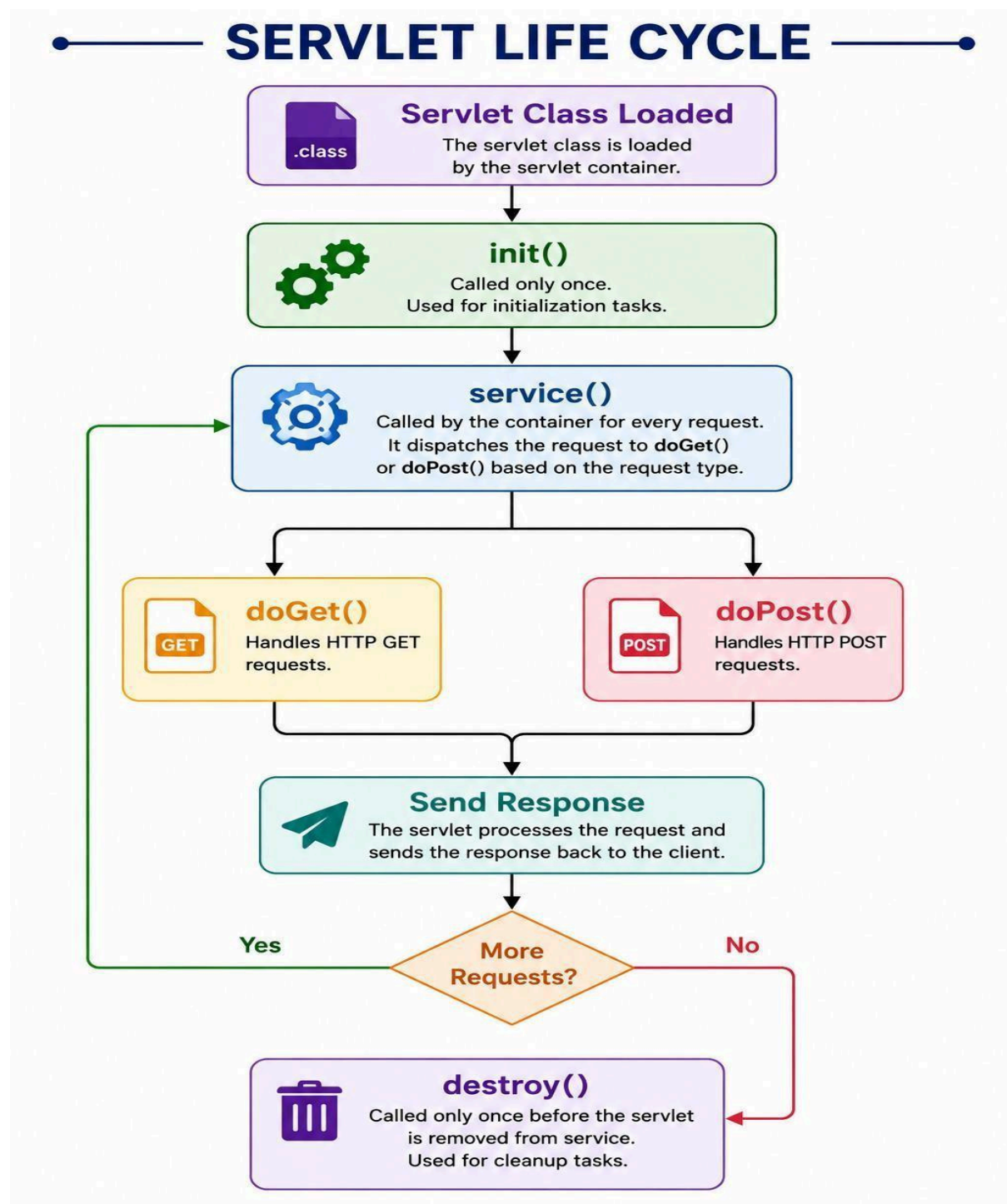
- Life cycle phases
- Execution flow
- Performance implications

Answer:

Servlet Life Cycle Analysis

The **Servlet Life Cycle** describes how a servlet is created, used, and destroyed by the servlet container.

Life Cycle Diagram



init()

The `init()` method is called **once** when the servlet is initialized.

```
public void init() {
```

```
    System.out.println("Servlet initialized");
```

}

Uses

- Initialize resources
- Read configuration
- Establish required initial resources

service()

The service() method is called whenever the servlet receives a request.

For HttpServlet, it dispatches the request to methods such as:

GET → doGet()

POST → doPost()

Example:

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response) {
    // Process GET request
}
protected void doPost(HttpServletRequest request,
                    HttpServletResponse response) {
    // Process POST request
}
```

destroy()

The destroy() method is called **once** before the servlet is removed from service.

```
public void destroy() {
    System.out.println("Servlet destroyed");
}
```

Uses

- Release resources
- Close connections
- Perform cleanup

Performance Implications

Method	Frequency	Performance
init()	Once	Expensive initialization can be done here
service()	Every request	Should be efficient
destroy()	Once	Used for cleanup

Important: A servlet instance can handle multiple requests, potentially concurrently. Therefore, shared mutable instance variables should be handled carefully.

3. GET vs POST Method

Evaluation Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations

Answer:

GET vs POST Method Evaluation

Both GET and POST are HTTP methods commonly handled by servlets.

GET

GET sends parameters as part of the URL.

Example:

Name:

Servlet:

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
                    throws ServletException, IOException {
    String name = request.getParameter("name");
    response.getWriter().println(
        "Welcome " + name
    );
}
```

Suitable Uses

- Searching
- Filtering
- Retrieving information
- Pages that can be bookmarked
- Read-only operations

Example:

/products?category=mobile

POST

POST sends data in the HTTP request body.



A web form for registration. It features two input fields: 'Name:' followed by a text box, and 'Email:' followed by a text box. To the right of the email field is a button labeled 'Register'. The form is set against a light gray background.

Servlet:

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    String email = request.getParameter("email");

    response.getWriter().println(
        "Registration successful"
    );
}
```

Suitable Uses

- Registration
- Login
- Payment/order submission
- Updating information
- Uploading data

GET vs POST Table

Feature	GET	POST
Data location	URL/query string	Request body
Visibility	Visible in URL	Not normally visible in URL
Bookmarking	Possible	Generally not used for bookmarking
Caching	Can be cached	Generally not treated as a simple cacheable retrieval
Data size	Limited by URL/request-line limits	Supports larger request bodies
Idempotency	Intended for safe retrieval	Often used for state-changing operations
Sensitive data	Not appropriate	More appropriate, but HTTPS is still required
Typical use	Search/retrieve	Submit/create/update

Security Point

POST is preferable to GET for passwords and other sensitive form data because the data is not placed in the URL. However, POST does not encrypt the data. HTTPS/TLS is required for confidentiality.

4. Session Tracking

Techniques Question:

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

Answer:

Session Tracking Techniques

HTTP is **stateless**, so a web application needs session tracking to recognize the same user across multiple requests.

The major techniques are:

1. HttpSession
2. Cookies
3. URL Rewriting

A. HttpSession

HttpSession stores session-related information on the server.

Example

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username",
```

```
"Sravanthi"); Retrieve it:
```

```
HttpSession session =  
    request.getSession(false);
```

```
String username =  
    (String)
```

```
session.getAttribute("username"); Logout:
```

```
session.invalidate();
```

Advantages

- Convenient API
- Session data is maintained server-side
- Good for login/authentication state
- Can store multiple attributes

Limitations

- Uses server memory/storage
- Requires session management infrastructure
- Session IDs must be protected

B. Cookies

Cookies are small pieces of data stored by the browser.

Create Cookie

```
Cookie cookie =  
    new Cookie("username", "Sravanthi");
```

```
cookie.setMaxAge(3600);

response.addCookie(cookie);

Read Cookie
Cookie[] cookies =
    request.getCookies();

if (cookies != null) {

    for (Cookie c : cookies) {

        if (c.getName().equals("username")) {

            System.out.println(c.getValue());
        }
    }
}
```

Advantages

- Simple
- Can persist beyond a single session
- Useful for preferences and small client-side values

Limitations

- Stored on the client
- Users can delete or modify them
- Limited storage
- Should not contain sensitive information directly

C. URL Rewriting

URL rewriting adds the session identifier to the URL when cookies cannot be used.

Example:

`https://example.com/profile;jsessionid=ABC123`

Servlet example:

```
String url =
    response.encodeURL("profile");
```

```
out.println("<a href='" + url +  
            "'>Profile</a>");
```

Advantages

- Works when cookies are disabled
- Useful as a fallback session-tracking mechanism

Limitations

- Session ID can appear in URLs
- URLs may be logged or shared
- Less convenient
- Requires links to be encoded properly

Session Tracking Comparison

Technique	Storage	Advantage	Limitation
HttpSession	Server	Good for login/session data	Uses server resources
Cookies	Client	Simple and persistent	Can be modified/deleted
URL Rewriting	URL + server session	Works without cookies	Session ID may leak through URLs

Best Choice

For a login system:

HttpSession
+
Secure session cookie
+
HTTP
S

is generally the preferred approach



Session Tracking Techniques

[Preview](#)

[HttpSession](#) • [Cookies](#) • [URL Rewriting](#)



HttpSession

Maintains user information on the server across multiple requests.

Advantages

- Server-side storage
- Easy to use
- Suitable for login systems
- Can store multiple attributes

Best Use:
Online banking, student portals, e-commerce login.



Cookies

Small pieces of data stored in the user's browser.

Advantages

- Easy to implement
- Can persist for a long time
- Useful for preferences
- Reduces server-side storage

Best Use:
Language preferences, themes, remembering settings.



URL Rewriting

Session information is included in the URL.

Advantages

- Works without cookies
- Simple fallback mechanism
- Useful when cookies are disabled

Example:
profile;sessionId=ABC123

Limitation:
Session ID may appear in URLs.

Feature	HttpSession	Cookies	URL Rewriting
Storage	Server	Browser	URL + Server Session
Security	Higher	Medium	Lower
Works without Cookies	With URL fallback	✗	✓
Best For	Login Sessions	Preferences	Cookie-disabled Clients

5. Cookies vs Sessions – Critical

Analysis Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

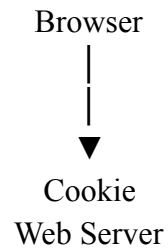
- Storage location
- Security risks
- Performance considerations

Answer:

Cookies vs Sessions – Critical Analysis

Cookies

A cookie is stored in the **user's browser**.



Example:

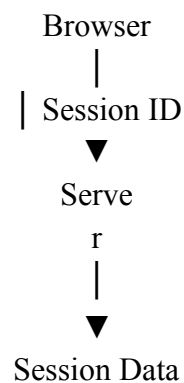
```

Cookie cookie =
    new Cookie("theme", "dark");

response.addCookie(cookie);
  
```

Sessions

A session stores the user's application state primarily on the **server**.



Example:

```

HttpSession session =
    request.getSession();

session.setAttribute(
    "username",
    "Sravanthi"
);
  
```

Cookies vs Sessions

Feature	Cookies	Sessions
Storage	Client/browser	Mainly server
Identification	Cookie value	Session ID
Size	Small	Depends on server resources

Persistence	Can persist for a specified time	Usually expires after timeout/invalidation
-------------	----------------------------------	--

Feature	Cookies	Sessions
Security	Client can inspect/modify values	Data is server-side
Server memory	Low	Uses server resources
Performance	Avoids server-side storage for simple values	Requires server-side session lookup
Best use	Preferences, non-sensitive identifiers	Login/authentication and user state

Real-Time Use Cases

1. Online Shopping

Cookies can remember:

Language = English
Theme = Dark

A session can maintain:

Logged-in user
Shopping cart
Checkout state

2. Banking Website

Session management can maintain:

Authenticated user
Current login session
Authorization state

Sensitive information should not simply be stored in client-side cookies.

3. Social Media

A session can maintain the user's authenticated state across:

Home
↓

Profile



Message

s

↓
Settings

4. E-Commerce Preferences

Cookies can remember:

Preferred language
Recently selected preferences
Security Considerations

For authentication sessions, applications should use secure cookie attributes such as:

Secure

HttpOnly
SameSite

Secure

Cookie is sent only over HTTPS.

HttpOnly

Helps prevent JavaScript from directly reading the cookie.

SameSite

Helps reduce certain cross-site request risks.

Also:

- Use HTTPS.
- Do not store passwords in cookies.
- Use sufficiently strong, unpredictable session IDs.
- Invalidate sessions during logout.
- Regenerate the session ID after authentication where appropriate.

Cookies vs Sessions

Critical Analysis of Web Application Session Management



Cookies

Storage Location

User's web browser

Security

Client-side data can be viewed or modified.

Performance

Reduces server-side storage for simple values.

Persistence

Can remain in the browser until expiration.

Best Use

Preferences, language, theme and small non-sensitive values.



Sessions

Storage Location

Mainly stored on the server.

Security

Application state remains on the server; the browser generally holds only a session ID.

Performance

Uses server resources to maintain session state.

Persistence

Usually ends after timeout or invalidation.

Best Use

Login authentication, shopping carts, and user-specific application state.

Critical Comparison

Feature	 Cookies	 Sessions
Storage	Client / Browser	Server
Data Visibility	Visible to Client	Primarily Server-side
Server Resource Usage	Low	Higher
Persistence	Can be long-term	Usually temporary
Modification by User	Possible	Session data is server-side
Login Management	Usually carries session ID	Stores login state
Suitable for Sensitive State	 Avoid storing sensitive data	 Better suited

Real-Time Use Cases

E-Commerce

Cookie: Remember language and preferences.

Session: Maintain login and shopping cart.

Banking

Cookie: Can support session identification.

Session: Maintain authenticated user state.

Social Media

Cookie: Remember preferences.

Session: Maintain logged-in state.

Security Recommendations

- Use HTTPS for authenticated sessions.
- Do not store passwords in cookies.
- Use Secure cookies when using HTTPS.
- Use HttpOnly to reduce JavaScript access to cookies.
- Use SameSite appropriately.
- Invalidate sessions during logout.

Code of Conduct:

I _____ (K.Deepika / 192371042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Deepika

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness